

HW05 — Performance Testing · Main Report

Student ID: 23127249 · **Tool:** Apache JMeter 5.6.3 · **AI:** Claude (Opus 4.8) **SUT:** EShop backend (Node.js + SQLite) — `http://localhost:3000` **Machine:** ThieuNagg — i5-12500H, 16 GB, Windows 11 (26200), JDK 25 **GitHub repo:** `https://github.com/phucnotphuc/eshop-hw05-performance` **Demo video (unlisted):**

1. Scope — endpoint selection

One end-to-end workflow, reused across all three scenarios, covering all three groups:

```
login (auth) → search products + product detail (read) → add-to-cart + apply-coupon + checkout
```

Group	FR	API	Justification
Auth-heavy	FR20	<code>POST /api/login</code>	JWT issuance; account-lockout rule (3 fails / 30s) on wrong password
Read-heavy	FR05/FR15	<code>GET /api/products?search=</code> , <code>GET /api/products/:id</code>	read-dominant, cache/DB-read bound
Transactional	FR07/08/09	<code>POST /api/cart</code> , <code>POST /api/apply-coupon</code> , <code>POST /api/checkout</code>	writes + coupon calc; latency-sensitive path

Selection matches my HW02 features (FR04/FR09/FR15/FR20) and is not duplicated with group members.

2. Test design (AI-assisted, step-by-step)

The plans were generated by driving the AI step by step (endpoint mapping → CSV design → base `.jmx` → scenario variants), **not** with a single generic prompt. Full prompt/response log in the AI Audit Report appendix. Generator: `test-plans/generate_plans.js` (parameterized, reproducible).

Data-driven inputs (CSV): - `data/users.csv` — 50-user pool (pre-registered via `POST /api/register`). - `data/products.csv` — 5 SKUs (id, name, price, search term). - `data/coupons.csv` — SAVE10 / BIGBUY / VIP100 with thresholds.

Scenario parameters (threads/ramp/duration exposed via `${__P(...)}`):

Scenario	Threads (VU)	Ramp-up	Duration	Think-time	Listener (distinct)	File
Load	50	60 s	10 min	1–2 s	Summary Report	<code>23127249_Load_20260830.jmx</code>
Stress	ramp→300	180 s	6 min	0.3–0.7 s	Aggregate Report	<code>23127249_Stress_20260830.jmx</code>

Scenario	Threads (VU)	Ramp-up	Duration	Think-time	Listener (distinct)	File
Spike	200 burst	3 s	90 s	0.1–0.3 s	View Results Tree	23127249_Spike_20260830.jmx
Soak (endurance)	40	30 s	12 min	1–2 s	(reuse)	Load plan, -JDURATION=720

Three distinct report views (Summary / Aggregate / View Results Tree) — no type repeated. HTML dashboards additionally generated via `-e -o` for every run.

Parameter justification: think-time models real user pauses so throughput isn't unrealistically inflated; Load ramp (60 s) avoids a cold-start thundering herd; Stress ramps gradually to 300 to locate the knee; Spike uses a near-instant 3 s ramp to test burst recovery.

3. Human review — what the AI got wrong / missed

Critical review of the first AI-generated plan and the corrections I made (I am responsible for the final plans):

#	AI mistake / omission	Correction	Why the AI missed it
1	Guessed coupon endpoint as <code>/api/coupons/apply</code>	Real path is <code>POST /api/apply-coupon</code>	Model inferred from REST convention instead of reading the SUT's <code>api_specification.md</code>
2	Extracted only the JWT <code>token</code> from login	Also extract <code>user.id</code> → needed by <code>apply-coupon</code> body	Prompt didn't state the coupon body needs <code>user_id</code> ; model didn't cross-read the two specs
3	Reused a single seed account for all VUs	Pre-registered a 50-user pool via CSV	Only 2 users are seeded; a single user gives unrealistic cache locality and risks lockout
4	No <code>Authorization</code> header on <code>cart/coupon/checkout</code>	Added a Bearer HeaderManager scoped to the transactional samplers	Model treated all samplers uniformly; missed that only these require auth
5	Averages-only reporting suggestion	Report p95/p99 for tail latency	LLMs default to "average latency" framing; averages hide the tail
6	Optimistic think-time (near-zero) for Load	Set realistic 1–2 s think-time for Load	Model optimizes for high throughput numbers, not realism
7	(Tooling) piping JMeter stdout through <code>head</code>	Capture full output, filter after	SIGPIPE killed the run early — an environment issue, not a plan flaw

Account-lockout handling: the workflow logs in with valid pooled credentials, so the 3-fail/30 s lockout does **not** trigger during Load/Stress/Spike. Reset procedure documented in the runbook for the case of deliberate wrong-password testing.

4. Execution & results

Numbers below are read from the raw `.jtl` via `test-plans/analyze_jtl.js` (authoritative). Resource-monitor screenshots: `evidence/resource-*.png`. HTML dashboards: `results/html-report-*`.

Overall (from `analyze_jt1.js` , authoritative — all times in ms):

Scenario	Samples	Window	Throughput	Error %	avg	p50	p90	p95	p99	max
Load (50 VU)	3 774	592 s	6.37 req/s	0.00	3	3	7	8	10	31
Stress (→300 VU)	32 215	357 s	90.15 req/s	0.00	3	2	7	9	14	67
Spike (200 VU burst)	17 453	89 s	195.80 req/s	0.00	5	3	10	13	19	38
Soak (40 VU, 12 min)	3 741	712 s	5.25 req/s	0.00	3	3	6	7	10	33

4.1 Load (`23127249_Load_20260830` — Summary Report)

50 VU, 60 s ramp, 10 min. **0 errors / 3 774 samples**. The transactional `Checkout` is the slowest step (avg 8 ms, p95 10 ms, max 18 ms); reads/login are ~2–3 ms. p95 overall 8 ms.

Step	n	err%	avg	p95	max
01 Login (auth)	649	0	3	5	31
02 Search (read)	639	0	3	4	16
03 Detail (read)	630	0	2	4	12
04 Add-to-cart (txn)	626	0	2	3	7
05 Apply-coupon (txn)	618	0	3	5	17
06 Checkout (txn)	612	0	8	10	18

4.2 Stress (`23127249_Stress_20260830` — Aggregate Report)

Ramp 0→300 VU over 180 s, 6 min. **0 errors / 32 215 samples**, throughput 90 req/s. Latency barely rose: overall p95 9 ms, worst single sample 67 ms (checkout). The backend did **not** reach a breaking point — no knee in latency, no errors — so 300 VU is within capacity on this hardware. Checkout p95 12 ms, login p95 9 ms.

4.3 Spike (`23127249_Spike_20260830` — View Results Tree)

200 VU injected in 3 s, held 90 s. **0 errors / 17 453 samples**, peak throughput 196 req/s (highest of all scenarios). The instantaneous burst pushed p95 to 13 ms and login p95 to 13 ms (the auth path feels the burst most), but the system absorbed and recovered with zero failures.

4.4 Endurance / soak threshold

Load plan at 40 VU sustained for **11.9 minutes** (`-JDURATION=720`). Result over the full window: **0 % errors, throughput 5.25 req/s, p95 7 ms, p99 10 ms** — no latency drift, no error creep. Backend `node.exe` resource timeline (`evidence/resource_timeline.png` , sampled every 2 s):

- **Memory ceiling:** ~72 MB working set (started ~45 MB, stabilised 65–72 MB — **no leak** over 12 min).
- **CPU:** negligible (<1–2 %); the backend is not CPU-bound.
- **Peak threads:** 32.

Threshold conclusion (with numbers): on `ThieuNagg` (i5-12500H, 16 GB) the EShop backend sustained **~90 req/s at 300 VU** and a **~196 req/s burst** with **0 % errors and p95 ≤ 16 ms**, memory capped at **~72 MB**. The empirical limit reached in these tests was the **JMeter client + think-time**, not the server — the SUT (single-node SQLite, 5-product

dataset, sub-ms queries) has headroom beyond what one JMeter instance on the same machine could generate. To truly saturate it would require a distributed JMeter setup or a much larger dataset; documented as a limitation.

5. Task 2 — AI analysis + misinterpretation hunt

See `ai/AI-Analysis-Critique.md`.

6. Task 3 — Continuous Performance Testing proposal

See `report/CPT-Proposal.md` (flow chart + trade-offs).

7. Bugs / issues

See `ai/Bug-Report.md` and the SUT GitHub Issues (links + screenshots).

Appendix A — AI Audit Report

See `ai/AI-Audit-Report.md`.

Appendix B — AI Critique (200–300 words)

See `ai/AI-Critique.md`.